

Software Requirements Specification (SRS)

AI-Powered MPLADS Monitoring & Anomaly Detection System

Smart India Hackathon 2026 — Problem Statement PS 26102
Team: Code Blooded | Standard: IEEE Std 830-1998 / ISO 29148:2018
Version: 1.0.0 | Date: September 2026 | Status: Implemented & Verified

Document Control & Revision History

Version	Date	Description	Author	Approved By
1.0.0	09-09-2026	Initial baseline SRS reflecting the implemented production system (FastAPI, PostgreSQL, React 19, 4 Independent Analytical Engines).	Team Code Blooded	Lead Architect

Table of Contents

1. [Introduction](#1-introduction)

• 1.1 [Purpose](#11-purpose)

• 1.2 [Scope](#12-scope)

• 1.3 [Intended Audience & Reading Suggestions](#13-intended-audience--reading-suggestions)

• 1.4 [Definitions, Acronyms, and Abbreviations](#14-definitions-acronyms-and-abbreviations)

• 1.5 [References](#15-references)

2. [Overall Description](#2-overall-description)

• 2.1 [Product Perspective](#21-product-perspective)

• 2.2 [Product Objectives](#22-product-objectives)

• 2.3 [Major System Capabilities](#23-major-system-capabilities)

• 2.4 [User Classes and Stakeholder Roles](#24-user-classes-and-stakeholder-roles)

• 2.5 [Operating Environment](#25-operating-environment)

• 2.6 [Design and Implementation Constraints](#26-design-and-implementation-constraints)

• 2.7 [Assumptions and Dependencies](#27-assumptions-and-dependencies)

3. [System Architecture](#3-system-architecture)
 - 3.1 [High-Level Architectural Model](#31-high-level-architectural-model)
 - 3.2 [Frontend Architecture (React 19 / TypeScript)](#32-frontend-architecture-react-19--typescript)
 - 3.3 [Backend & API Architecture (FastAPI)](#33-backend--api-architecture-fastapi)
 - 3.4 [Database & Storage Layer (PostgreSQL on Supabase)](#34-database--storage-layer-postgresql-on-supabase)
 - 3.5 [Authentication, Scoping & Security Boundary](#35-authentication-scoping--security-boundary)
 - 3.6 [Analytics & Machine Learning Processing Layer](#36-analytics--machine-learning-processing-layer)
 - 3.7 [Data Flow and Component Interactions](#37-data-flow-and-component-interactions)
4. [Functional Requirements](#4-functional-requirements)
 - 4.1 [Authentication & Session Lifecycle (FR-01, FR-02)](#41-authentication--session-lifecycle-fr-01-fr-02)
 - 4.2 [Server-Side Jurisdictional RBAC (FR-03)](#42-server-side-jurisdictional-rbac-fr-03)
 - 4.3 [Role-Adaptive Governance Dashboards (FR-04 to FR-07)](#43-role-adaptive-governance-dashboards-fr-04-to-fr-07)
 - 4.4 [Works Master Registry & Filtering (FR-08)](#44-works-master-registry--filtering-fr-08)
 - 4.5 [Single-Work Investigation Dossier (FR-09)](#45-single-work-investigation-dossier-fr-09)
 - 4.6 [Independent Analytical Consoles (FR-10 to FR-14)](#46-independent-analytical-consoles-fr-10-to-fr-14)
 - 4.7 [Administrative Summaries (FR-15, FR-16)](#47-administrative-summaries-fr-15-fr-16)
 - 4.8 [Ministry User Administration (FR-17)](#48-ministry-user-administration-fr-17)
 - 4.9 [System Health & Infrastructure Telemetry (FR-18)](#49-system-health--infrastructure-telemetry-fr-18)
5. [Analytics & Machine Learning Specifications](#5-analytics--machine-learning-specifications)
 - 5.1 [Model 1: Anomalous Cost Estimate Detector (Isolation Forest)](#51-model-1-anomalous-cost-estimate-detector-isolation-forest)
 - 5.2 [Model 2: Potential Duplicate Work Detector (NLP Embeddings & Blocking)](#52-model-2-potential-duplicate-work-detector-nlp-embeddings--blocking)
 - 5.3 [Model 3: Fund & Expenditure Anomaly Detector (Isolation Forest & Rules)](#53-model-3-fund--expenditure-anomaly-detector-isolation-forest--rules)
 - 5.4 [Model 4: Statutory Delay & SLA Tracking Engine (Deterministic Rules)](#54-model-4-statutory-delay--sla-tracking-engine-deterministic-rules)
 - 5.5 [Zero Composite Scoring Policy](#55-zero-composite-scoring-policy)
6. [Data Requirements](#6-data-requirements)
 - 6.1 [Source Data Acquisition & Lineage](#61-source-data-acquisition--lineage)
 - 6.2 [Canonical Normalization & Transformation Pipeline](#62-canonical-normalization--transformation-pipeline)
 - 6.3 [Data Validation, Edge Cases & Data Quality Exceptions](#63-data-validation-edge-cases--data-quality-exceptions)
7. [Database Schema & Entity Specifications](#7-database-schema--entity-specifications)
 - 7.1 [Relational Schema Overview](#71-relational-schema-overview)
 - 7.2 [Table Definitions & Constraint Rules](#72-table-definitions--constraint-rules)

8. [API Interface Specifications](#8-api-interface-specifications)
 - 8.1 [Authentication Endpoints](#81-authentication-endpoints)
 - 8.2 [Works Master Registry Endpoints](#82-works-master-registry-endpoints)
 - 8.3 [Analytical Module Endpoints](#83-analytical-module-endpoints)
 - 8.4 [Governance Summary Endpoints](#84-governance-summary-endpoints)
 - 8.5 [Health & Diagnostics Endpoint](#85-health--diagnostics-endpoint)
9. [Security & Access Control Requirements](#9-security--access-control-requirements)
10. [User Interface Specifications](#10-user-interface-specifications)
11. [Non-Functional Requirements](#11-non-functional-requirements)
12. [Error Handling & Edge Cases](#12-error-handling--edge-cases)
13. [Requirements Traceability Matrix](#13-requirements-traceability-matrix)
14. [Known System Limitations](#14-known-system-limitations)
15. [Future Enhancements (Deferred Scope)](#15-future-enhancements-deferred-scope)
16. [Acceptance Criteria & Test Verification](#16-acceptance-criteria--test-verification)
-

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) establishes the complete, authoritative technical specification for the **AI-Powered MPLADS Monitoring & Anomaly Detection System**, engineered by Team **Code Blooded** for Smart India Hackathon (SIH) 2026 Problem Statement **PS 26102**.

This document describes the actual implemented production system. Every requirement, schema, API endpoint, machine learning model, rule engine, and user interface component documented herein has been verified against the production codebase and its test suite.

1.2 Scope

The system is an enterprise-grade, full-stack intelligence and monitoring platform designed for the **Ministry of Statistics and Programme Implementation (MoSPI)**, State Nodal Authorities, District Authorities, and Hon'ble Members of Parliament.

The system processes **98,825 official works** and **4,286+ Crore** in sanctioned public capital across **36 States/UTs** and **767 administrative districts**. It performs automated, independent anomaly audits across four operational dimensions:

1. **Cost Anomaly Detection** (peer-group calibrated outlier identification via Isolation Forest).
2. **Potential Duplicate Work Detection** (semantic sentence embeddings with blocking windows).
3. **Fund Expenditure & Vendor Concentration Analysis** (disbursement pacing, dormant sanctions, and Herfindahl-Hirschman Index payment concentration).
4. **Statutory Delay & SLA Tracking** (deterministic enforcement of official MoSPI 2023 Guidelines: 75-day sanction SLA and 365-day execution threshold).

The platform strictly isolates data access across four administrative tiers through server-side jurisdictional predicate injection, backed by an asynchronous FastAPI REST backend, a PostgreSQL relational database on Supabase, and a responsive React 19 / TypeScript single-page application.

1.3 Intended Audience & Reading Suggestions

- **SIH 2026 Evaluation Committee & Technical Jury**: Focus on Sections 2, 3, 4, 5, 13, and 16 to evaluate compliance with PS 26102.
- **Government Administrative Authorities (MoSPI / District Collectors)**: Focus on Sections 2.4, 4, 5, and 10 to inspect user workflows, dashboards, and analytical defensibility.
- **Software Engineers & DevOps**: Focus on Sections 3, 7, 8, 9, 11, and 12 for technical maintenance, deployment, and API integration.
- **Data Scientists & Machine Learning Engineers**: Focus on Section 5 for mathematical formulation, feature engineering, and peer-group hierarchy.

1.4 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
MPLADS	Members of Parliament Local Area Development Scheme (Central Sector Scheme).
MoSPI	Ministry of Statistics and Programme Implementation, Government of India.
IDA	Implementing District Authority (the administrative authority responsible for executing works).
HHI	Herfindahl-Hirschman Index: Statistical metric measuring vendor/payee concentration ($\sum s_i^2$).
SLA	Service Level Agreement: Legally binding timeline milestones codified in MPLADS Guidelines 2023.
RBAC	Role-Based Access Control: Security model restricting system operations based on authenticated roles.
JWT	JSON Web Token (RFC 7519): Compact, URL-safe means of representing claims signed via HMAC-SHA256.
MiniLM	`sentence-transformers/all-MiniLM-L6-v2`: 384-dimensional dense sentence transformer embedding model.
Isolation Forest	Unsupervised ensemble learning algorithm isolating anomalies by partitioning feature space.
Zero Composite Score	Architectural principle barring synthetic formulaic combinations of distinct analytical models.
Dormant Sanction	Work sanctioned >180 days prior with zero financial disbursement records.
Potential Duplicate	Statistical pair-level candidate exhibiting high semantic/spatial/temporal overlap requiring human review.

1.5 References

1. **MPLADS Guidelines 2023**, Ministry of Statistics and Programme Implementation, Government of India (Effective April 1, 2023).

2. **SIH 2026 Problem Statement PS 26102**: "Development of an AI-powered system to detect anomalies, fraud, and inefficiencies in MPLAD Scheme implementation".
3. **IEEE Std 830-1998**: IEEE Recommended Practice for Software Requirements Specifications*.
4. **RFC 7519**: JSON Web Token (JWT) Architecture and Specifications*.
5. **RFC 6749**: The OAuth 2.0 Authorization Framework (Bearer Token Usage)*.

2. Overall Description

2.1 Product Perspective

The AI-Powered MPLADS Monitoring System operates as a modern, self-contained analytical and investigative suite. Prior to this system, scheme oversight relied on retrospective manual sampling, fragmented spreadsheets, and post-audit PAC (Public Accounts Committee) queries often raised years after project completion.

This system interfaces directly with the national work registry and expenditure transaction logs. It is deployed as a decoupled multi-tier architecture:

- **Presentation Layer**: Client-side React 19 single-page application communicating via REST/JSON.
- **Application Layer**: Asynchronous FastAPI service running on Uvicorn with SlowAPI rate limiting.
- **Analytical Layer**: Offline/batch feature engineering and model training pipelines persisting calibrated results.
- **Persistence Layer**: Relational PostgreSQL database managed via SQLAlchemy ORM.

2.2 Product Objectives

1. **Proactive Anomaly Detection**: Identify inflated cost estimates prior to fund exhaustion using localized historical peer distributions.
2. **Ghost Work & Duplicate Prevention**: Block redundant allocations across multiple financial years or sitting MPs through dense semantic matching.
3. **Vendor Cartelization Mitigation**: Flag disproportionate vendor concentration using Herfindahl-Hirschman Indexing.
4. **Statutory SLA Enforcement**: Eliminate administrative bottlenecks by systematically flagging delays exceeding the 75-day sanction window and 365-day completion target.
5. **Constitutional & Jurisdictional Privacy**: Prevent unauthorized horizontal or vertical data leakage across administrative divisions.
6. **Legally Defensible Explainability**: Provide natural language justifications for every flagged work with zero synthetic composite black-box scoring.

2.3 Major System Capabilities

- **Role-Adaptive Visual Interfaces**: Dynamically configured dashboards tailored to Ministry executives, State Nodal Officers, District Collectors, and Members of Parliament.
- **Searchable Master Catalog**: Real-time multi-parameter querying across 98,825 records with sub-second paginated responses.
- **Single-Work Investigative Dossier**: Unified dossier displaying full metadata, milestone chronology, vendor vouchers, and decoupled risk profile cards across all 4 analytical models.

- ****Side-by-Side Duplicate Comparison Console****: Visual inspection workspace providing token diffs, coordinate/date proximity, and cross-jurisdictional data redaction.
- ****Server-Side Predicate Injection****: Enforcement of data boundaries at the SQL planner level, ensuring unauthorized records cannot be extracted even through direct API manipulation.

2.4 User Classes and Stakeholder Roles

The system supports four administrative and constitutional stakeholder roles:

+-----+

MINISTRY
National Scope: All 36 States/UTs, 767 Districts
Full User Provisioning & Global Analytics

+-----+

V

+-----+

STATE_OFFICER
State Scope: All Districts in Assigned State
Inter-District Governance & Monitoring

+-----+

V

+-----+

DISTRICT_OFFICER
District Scope: Assigned State + Assigned District
Operational Triage & Voucher Investigation

+-----+

V

+-----+

MP
MP Scope: Works Recommended by Specific MP Name
Constituency Portfolio & Lifecycle Tracking

+-----+

Role Identifier	Representative Stakeholder	Geographical / Jurisdictional Scope	Access Boundary Implementation
-----------------	----------------------------	-------------------------------------	--------------------------------

MINISTRY	Central Ministry (MoSPI), Joint Secretary, Audit Officers	Entire National Territory (36 States/UTs, 767 Districts, 98,825 works)	Unrestricted. Global database access; exclusive access to user management (`/admin/users`).
STATE_OFFICER	State Nodal Authority, State Planning Department	Specific Assigned State (e.g., Uttar Pradesh)	SQL Predicate: `WHERE works.state = :assigned_state`. Cannot access works of other States.
DISTRICT_OFFICER	District Magistrate (DM), Deputy Commissioner (DC), IDA	Specific Assigned District + State (e.g., Patna, Bihar)	SQL Predicate: `WHERE works.state = :assigned_state AND works.district = :assigned_district`.
MP	Hon'ble Member of Parliament (Lok Sabha / Rajya Sabha)	Works recommended by the specific MP Name	SQL Predicate: `WHERE works.mp_name = :assigned_mp_name`. Cannot view portfolio of peer MPs.

2.5 Operating Environment

- ****Server Runtime****: Python 3.12.x on Windows/Linux environments.
- ****ASGI Web Server****: Uvicorn 0.52+ running FastAPI 0.141+.
- ****Database Engine****: PostgreSQL 17+ hosted on Supabase Cloud (with connection pooling).
- ****Client Runtime****: Modern web browsers (Google Chrome 110+, Microsoft Edge 110+, Mozilla Firefox 110+, Safari 16+) with JavaScript enabled.
- ****Frontend Tooling****: Node.js v22.x, Vite 8.2, React 19.2.

2.6 Design and Implementation Constraints

1. ****Statutory Decoupling (Zero Composite Score Constraint)****:

In administrative law and government oversight, amalgamating distinct allegations (e.g., a cost variance and a procedural delay) into a single composite index creates legally vulnerable accusations and obscures root causes. The system strictly prohibits synthetic composite risk scores.

2. ****Dual-Column District Scoping Constraint****:

Across India, 75 district names exist in multiple states (e.g., `BILASPUR` exists in Chhattisgarh and Himachal Pradesh; `PRATAPGARH` exists in Uttar Pradesh and Rajasthan; `AURANGABAD` exists in Maharashtra and Bihar). All district queries must strictly filter on the composite tuple `(state, district)`.

3. ****No Public Self-Registration Constraint****:

Because access grants constitutional and administrative authority over government records, self-registration is strictly barred. Accounts must be provisioned by the Ministry.

4. ****Offline Analytical Execution Constraint****:

Due to the computational intensity of calculating 84,796 dense text embeddings and training 1,200+ localized Isolation Forest peer models, model execution is decoupled from real-time API transactions. Models write to dedicated database result tables which the API reads with indexed sub-millisecond queries.

2.7 Assumptions and Dependencies

- ****Data Completeness****: It is assumed that official portal exports provide reliable work descriptions, sanction dates, and disbursement vouchers.

- ****Statutory Gap****: The existing national portal export does not capture the date of formal rejection communications for non-sanctioned proposals. Therefore, the 45-day statutory rejection SLA cannot be evaluated against historical data.

3. System Architecture

3.1 High-Level Architectural Model

+-----+

PRESENTATION TIER (SPA)
React 19 + TypeScript + Vite 8 + Tailwind CSS + TanStack Query v5 + React Router v7 + Recharts

+-----+

HTTP/JSON (REST) | Reverse Proxy (:5173 -> :8000)

Bearer JWT Auth |

v

+-----+

APPLICATION SERVICE TIER								
FastAPI 0.141 / Uvicorn								
	Authentication &		Server-Side Scoping		SlowAPI Rate		Pydantic v2	
	Bcrypt Hashing		Predicate Injection		Limiter (5/min)		Response Models	
	API Routers: /auth, /works, /analytics/cost-anomalies, /analytics/duplicate-works,							
	/analytics/fund-anomalies, /analytics/delays, /analytics/summaries, /health							

+-----+

SQLAlchemy 2.0 | Parameterized SQL

Core & ORM | Connection Pooling

v



ANALYTICAL & MACHINE LEARNING PIPELINES						
	Model 1: Cost Anomaly		Model 2: Duplicate Work		Model 3: Fund & Expenditure	
	Isolation Forest		Sentence-Transformers		Isolation Forest + Rules	
	Peer Group Hierarchy		all-MiniLM-L6-v2		HHI Vendor Concentration	
	Model 4: Statutory Delay Engine (Deterministic Rule-Based Implementation of MoSPI 2023 SLA)					



3.2 Frontend Architecture (React 19 / TypeScript)

The client application is organized into a modular, unidirectional data flow architecture:

- `src/types/`: Strict TypeScript interfaces mirroring backend Pydantic models 1:1.
- `src/lib/api-client.ts`: Pre-configured Axios instance with request/response interceptors that automatically attach `Authorization: Bearer `, clear storage upon HTTP 401, and broadcast custom events upon HTTP 429.
- `src/context/AuthContext.tsx`: Centralized authentication state tracking current user profile, login lifecycle, and quick-switching between canonical demonstration accounts.
- `src/components/layout/ProtectedRoute.tsx`: Declarative routing guard inspecting authenticated session state and role permissions, rendering an official 403 Forbidden banner if unauthorized.
- `src/components/common/DataTable.tsx`: High-performance generic data table supporting pagination, sorting, custom cell renderers, and responsive scrolling across tens of thousands of records.

3.3 Backend & API Architecture (FastAPI)

The backend service utilizes FastAPI's asynchronous dependency injection framework:

- **api/auth/security.py**: Cryptographic module executing 12-round bcrypt password hashing, dummy constant-time verification, and RFC 7519 JWT generation.
- **api/auth/scoping.py**: Security policy enforcement layer intercepting SQLAlchemy queries and appending mandatory jurisdictional filtering predicates before execution.
- **api/auth/dependencies.py**: Session validation pipeline extracting Bearer tokens, decoding JWT signatures, querying live database state to confirm account activation, and enforcing role whitelists (`require_roles`).

3.4 Database & Storage Layer (PostgreSQL on Supabase)

Data persistence is managed via an enterprise PostgreSQL schema:

- Primary key enforcement across all canonical entities.
- Referential integrity guaranteed via foreign keys with `ON DELETE CASCADE`.
- Secondary B-tree indexes deployed on all high-frequency filtering attributes (`state`, `district`, `mp_name`, `work_type`, `severity`, `duplicate_score`, `cost_anomaly_score`, `fund_anomaly_score`, `delay_score`).
- Native PostgreSQL array data types (`TEXT[]`) storing multi-valued tags such as `anomaly_reasons` and `active_delay_types`.

3.5 Authentication, Scoping & Security Boundary

The application maintains an impermeable security boundary between the client and server:

- Frontend role checks and route guards exist purely for UX navigation and visual accommodation.
- The server never relies on client claims. All jurisdictional filtering is re-evaluated per HTTP request using the cryptographically verified `user_id` stored in the JWT payload.
- Real-time DB lookup in `get_current_active_user` guarantees instantaneous revocation: if an officer's account is marked `is_active = False` in the database, subsequent requests fail immediately with HTTP 401 regardless of token expiration time.

3.6 Analytics & Machine Learning Processing Layer

The analytical layer consists of four decoupled offline pipelines:

1. **Ingestion & Canonicalization**: Cleans, parses, and normalizes raw parliamentary datasets into standardized formats (`data_pipeline/`).
2. **Feature Engineering**: Constructs localized peer distributions, text token caches, disbursement timelines, and delay metrics (`feature_engineering/`).
3. **Scoring Pipelines**: Runs isolated training and scoring workflows, persisting parquet/CSV artifacts (`ml_models/`, `rule_engines/`).
4. **Database Population**: Bulk-loads scored results into relational PostgreSQL tables via parameterized inserts (`database/ingest.py`).

3.7 Data Flow and Component Interactions

[User Action in Browser]



[React Component (TanStack Query)] (Checks Client Cache) [Cached Data Displayed]

■ (Cache Miss / Invalidation)



[Axios API Client] ■■(Injects Authorization: Bearer <JWT>)



[FastAPI Route Handler]



■■■ [SlowAPI Limiter] ■■■■■■■■■■(Exceeded?)■■■ [HTTP 429 Too Many Requests]



■■■ [JWT Security Validator] ■■■■(Invalid?)■■■ [HTTP 401 Unauthorized]



■■■ [Live DB User Check] ■■■■■■(Inactive?)■■■ [HTTP 401 Unauthorized]



■■■ [Jurisdictional Scoping] ■■(Injects WHERE state/district/mp SQL)



[SQLAlchemy Query Planner]



[PostgreSQL Database (Supabase)] ■■(Executes Indexed Scan)



[Pydantic v2 Serializer] ■■■■■■■■(Validates & Strips Internal Fields)



[HTTP 200 JSON Response] ■■■■■■■■ [Rendered in React Data Table / Charts]

4. Functional Requirements

4.1 Authentication & Session Lifecycle

FR-01: User Authentication & Token Issuance

- ****Description****: The system shall authenticate registered administrative officers using official email and password credentials, returning a cryptographically signed RFC 7519 JWT access token.
- ****Inputs****: JSON payload containing `email` (string) and `password` (string).

- **Processing**:

1. Sanitize and lowercase email string.
2. Query `users` table for matching email.
3. If user not found, execute `verify\_dummy\_password()` (constant-time 12-round dummy bcrypt computation) and return HTTP 401.
4. If user found, verify password hash against stored `hashed\_password` using `bcrypt.checkpw()`.
5. Verify that `is\_active` flag is `True`. If `False`, return HTTP 401 Unauthorized.
6. Generate signed JWT containing `sub` (user email), `id` (user ID), `role`, `assigned\_state`, `assigned\_district`, `assigned\_mp\_name`, and `exp` (issued time + 60 minutes).

- **Outputs**: HTTP 200 JSON payload: `{"access\_token": "", "token\_type": "bearer"}`.

- **Access Restrictions**: Public endpoint, subject to strict rate limiting (5 attempts/minute/IP).

FR-02: Real-Time Session Verification & User Profile

- **Description**: The system shall return the authenticated officer's live profile, administrative role, and jurisdictional bounds.

- **Inputs**: HTTP `Authorization: Bearer` header.

- **Processing**:

1. Validate JWT signature against server `SECRET\_KEY` using HMAC-SHA256 (`HS256`).
2. Extract `user\_id` from payload.
3. Query `users` table by primary key (`id`). If user inactive or missing, reject with HTTP 401.
 - **Outputs**: HTTP 200 JSON payload conforming to `UserRead` schema: `id`, `email`, `full\_name`, `role`, `assigned\_state`, `assigned\_district`, `assigned\_mp\_name`, `is\_active`.
 - **Access Restrictions**: Any authenticated user.

4.2 Server-Side Jurisdictional RBAC

FR-03: Jurisdictional SQL Predicate Injection

- **Description**: The system shall enforce data boundary isolation at the database query layer.

- **Inputs**: Authenticated `CurrentUser` session and target SQLAlchemy query object.

- **Processing**:

1. If `user.role == "MINISTRY"`: Return query unmodified.
 2. If `user.role == "STATE\_OFFICER"`: Append `.filter(Work.state == user.assigned\_state)`.
 3. If `user.role == "DISTRICT\_OFFICER"`: Append `.filter(Work.state == user.assigned\_state, Work.district == user.assigned\_district)`.
 4. If `user.role == "MP"`: Append `.filter(Work.mp\_name.ilike(user.assigned\_mp\_name))`.
- **Outputs**: Filtered SQLAlchemy query object guaranteed to access only authorized records.
 - **Access Restrictions**: Internal system security function applied to all data access routes.

4.3 Role-Adaptive Governance Dashboards

FR-04: Ministry National Governance Dashboard

- **Description**: The system shall render national-level aggregated KPIs, financial outlay distributions, and independent high-severity audit queues across India.
- **Inputs**: None (derived from national database aggregates).
- **Processing**: Aggregate total works (98,825), sanctioned funds (₹4,286+ Cr), state-level budget comparisons, and independent high-severity counts across all 4 models.
- **Outputs**: National KPI metric cards, Comparative State Outlay Bar Chart, and 4 Independent Audit quick-triage tables.
- **Access Restrictions**: `MINISTRY` role only.

FR-05: State Officer Performance Dashboard

- **Description**: The system shall render state-level aggregates and inter-district performance comparisons for the officer's assigned state.
- **Inputs**: Automatic filtering on `current\_user.assigned\_state`.
- **Processing**: Compute total sanctioned capital, total disbursements, and district-by-district breakdown within the assigned state.
- **Outputs**: State KPI cards, District Performance Ranking Table with outlay, disbursed funds, and active anomaly tallies.
- **Access Restrictions**: `STATE\_OFFICER` role (scoped to assigned state).

FR-06: District Officer Operational Triage Dashboard

- **Description**: The system shall render district-specific operational queues highlighting stalled projects, un-disbursed sanctions, and cost outliers.
- **Inputs**: Automatic filtering on `current\_user.assigned\_state` and `current\_user.assigned\_district`.
- **Processing**: Extract works within the district requiring immediate administrative intervention (e.g., works with \$>180\$ days delay or dormant sanctions).
- **Outputs**: District KPI summary cards, Urgent Triage Action Queue, and Stalled Works Breakdown.
- **Access Restrictions**: `DISTRICT\_OFFICER` role (scoped to assigned district and state).

FR-07: Member of Parliament Portfolio Dashboard

- **Description**: The system shall render a personal parliamentary tracking console tracking recommendations through their 5-stage project lifecycle.
- **Inputs**: Automatic filtering on `current\_user.assigned\_mp\_name`.
- **Processing**: Query all works recommended by the MP; aggregate total recommended capital, sanctioned amounts, disbursed funds, and calculate progress across the 5 canonical database statuses: *Recommended \$→\$ Sanctioned \$→\$ Ongoing \$→\$ Completed \$→\$ Closed*.
- **Outputs**: Portfolio KPIs, 5-Stage Lifecycle Funnel Chart, Overall Fund Utilization Percentage Gauge, and Recent Recommendations Status Table.
- **Access Restrictions**: `MP` role (scoped to assigned MP name).

4.4 Works Master Registry & Investigation

FR-08: Works Master Registry & Multi-Parameter Search

- **Description**: The system shall provide a searchable, paginated registry of all works accessible within the user's jurisdiction.
- **Inputs**: Query parameters: `page`, `page\_size`, `search` (keyword in description), `state`, `district`, `mp\_name`, `house`, `work\_category`, `work\_status`, `min\_sanction\_amount`, `max\_sanction\_amount`.
- **Processing**: Apply server-side jurisdictional filter $\rightarrow$ apply user search/filter predicates $\rightarrow$ compute total matching count $\rightarrow$ fetch paginated slice sorted by `sanction\_date DESC`.
- **Outputs**: HTTP 200 `PaginatedResponse[WorkListItem]` containing `items` array and `pagination` metadata (`total\_records`, `page`, `page\_size`, `total\_pages`, `has\_next`, `has\_prev`).
- **Access Restrictions**: All authenticated users (scoped by jurisdiction).

FR-09: Single-Work Investigation Dossier

- **Description**: The system shall generate a comprehensive forensic dossier for a specific work ID, displaying complete financial attributes, vendor vouchers, and decoupled risk profiles.
- **Inputs**: Path parameter `work\_id` (string).
- **Processing**:

1. Query `works` table by `work\_id`. If not found, return HTTP 404 Not Found.
 2. Execute `verify\_work\_jurisdiction(work, current\_user)`. If work belongs outside user's jurisdiction, abort immediately with HTTP 403 Forbidden.
 3. Fetch associated records from `work\_expenditures`, `cost\_anomaly\_results`, `duplicate\_work\_results`, `fund\_expenditure\_results`, and `delay\_results`.
 4. Assemble independent risk profile objects without composite scoring.
- **Outputs**: HTTP 200 JSON payload conforming to `WorkDetail` schema.
 - **Access Restrictions**: Authenticated users possessing jurisdictional authority over the work.

4.5 Independent Analytical Consoles

FR-10: Cost Anomaly Detection Console

- **Description**: The system shall list works flagged for anomalous cost estimates, displaying peer group context and severity ratings.
- **Inputs**: Query parameters: `severity`, `min\_score`, `state`, `district`, `page`, `page\_size`.
- **Outputs**: HTTP 200 `PaginatedResponse[CostAnomalyItem]`.
- **Access Restrictions**: All authenticated users (scoped by jurisdiction).

FR-11: Potential Duplicate Work Detection Console

- **Description**: The system shall list flagged potential duplicate pairs, displaying pair scores, semantic similarities, and confidence tiers.
- **Inputs**: Query parameters: `severity`, `min\_duplicate\_score`, `is\_same\_mp`, `is\_same\_constituency`, `page`, `page\_size`.
- **Processing**: Applies pair-level scoping: User can discover pairs where at least one work resides within their jurisdiction (`work\_1.state = user.state OR work\_2.state = user.state`).

- **Outputs**: HTTP 200 `PaginatedResponse[DuplicatePairItem]`.
- **Access Restrictions**: All authenticated users (scoped by jurisdiction).

FR-12: Side-by-Side Duplicate Pair Comparison Console

- **Description**: The system shall render an interactive side-by-side comparison workspace evaluating two flagged work IDs.
- **Inputs**: Path parameter `work\_id` or query pair `work\_id\_1`, `work\_id\_2`.
- **Processing**:

1. Fetch metadata for both works.
2. If a work is outside the user's jurisdiction, redact confidential details and display an official "Jurisdiction Restricted (403)" placeholder card.
3. If both authorized, render comparative token differences, date diffs, amount ratios, and spatial proximity indicators.
 - **Outputs**: Side-by-side comparative UI view.
 - **Access Restrictions**: Authenticated users.

FR-13: Fund & Expenditure Anomaly Console

- **Description**: The system shall list works with disbursement pacing anomalies, dormant sanctions, or high vendor concentration.
- **Inputs**: Query parameters: `severity`, `audit\_category`, `min\_score`, `min\_utilization`, `max\_utilization`, `state`, `district`, `page`, `page\_size`.
- **Outputs**: HTTP 200 `PaginatedResponse[FundAnomalyItem]`.
- **Access Restrictions**: All authenticated users (scoped by jurisdiction).

FR-14: Statutory Delay & SLA Tracking Console

- **Description**: The system shall list works violating statutory timeline milestones under MPLADS Guidelines 2023.
- **Inputs**: Query parameters: `severity`, `primary\_delay\_type`, `min\_days\_overdue`, `state`, `district`, `page`, `page\_size`.
- **Outputs**: HTTP 200 `PaginatedResponse[DelayItem]`.
- **Access Restrictions**: All authenticated users (scoped by jurisdiction).

4.6 Administrative Summaries & User Management

FR-15: District Administrative Summary & Aggregations

- **Description**: The system shall provide aggregated work counts, financial outlays, and independent high-severity flags grouped by district.
- **Inputs**: Query parameters: `state` (optional), `limit` (integer, default 50).
- **Outputs**: HTTP 200 JSON `List[DistrictSummaryItem]`.
- **Access Restrictions**: Authenticated users (scoped by jurisdiction).

FR-16: MP Governance Summary & Aggregations

- **Description**: The system shall provide aggregated recommendation volumes, total sanctioned capital, and independent anomaly counts grouped by MP.

- **Inputs**: Query parameters: `state` (optional), `house` (optional), `limit` (integer, default 50).
- **Outputs**: HTTP 200 JSON `List[MPSummaryItem]`.
- **Access Restrictions**: Authenticated users (scoped by jurisdiction).

FR-17: Ministry User Provisioning & Administration

- **Description**: The system shall allow Ministry administrators to provision new officer accounts and inspect existing users.
- **Inputs**: For listing: None. For creation: `UserCreate` payload (`email`, `password`, `full\_name`, `role`, `assigned\_state`, `assigned\_district`, `assigned\_mp\_name`).
- **Processing**:
 1. Verify caller has `MINISTRY` role. If not, reject with HTTP 403 Forbidden.
 2. Check if email already registered. If yes, reject with HTTP 400 Bad Request.
 3. Hash password using bcrypt (12 rounds).
 4. Insert record into `users` table.
- **Outputs**: HTTP 201 Created returning `UserRead` payload.
- **Access Restrictions**: `MINISTRY` role only.

FR-18: System Health Monitoring & Telemetry

- **Description**: The system shall provide an unauthenticated health check endpoint verifying database connectivity and reporting query round-trip latency.
- **Inputs**: None.
- **Processing**: Execute lightweight SQL query `SELECT COUNT(\*) FROM works` and calculate elapsed latency in milliseconds.
- **Outputs**: HTTP 200 JSON payload: `{ "status": "healthy", "database": "PostgreSQL on Supabase", "db\_latency\_ms": 98825, "version": "1.0.0" }`.
- **Access Restrictions**: Public.

5. Analytics & Machine Learning Specifications

+-----+

FOUR DECOUPLED INDEPENDENT ANALYTICAL ENGINES
(Strict Zero Composite Score: Independent Forensic Signals)

+-----+

Module	Underlying Engine & Algorithm	Core Evaluated Signals
--------	-------------------------------	------------------------

+-----+

Model 1: Cost Anomaly Detector	Machine Learning: Isolation Forest	log(sanction_amount) vs
	with Hierarchical Peer Fallback	Local Peer Distribution
+-----+-----+-----+		
Model 2: Potential Duplicate Work	NLP & Embeddings: Sentence-	Dense Semantic Cosine +
Detector	Transformers (all-MiniLM-L6-v2)	Spatial-Temporal Window
+-----+-----+-----+		
Model 3: Fund & Expenditure	Hybrid: Isolation Forest +	Utilization Ratio +
Anomaly Detector	Deterministic Audit Category Rules	HHI Vendor Concentration
+-----+-----+-----+		
Model 4: Statutory Delay Engine	Deterministic Rule-Based Engine	75-Day Sanction SLA +
	(MPLADS Guidelines 2023 Para 3.12)	365-Day Completion SLA
+-----+-----+-----+		

5.1 Model 1: Anomalous Cost Estimate Detector (Isolation Forest)

- **Purpose**: Detect works with abnormally high or low sanctioned cost estimates compared to historical peer projects of identical type in the same geographical jurisdiction.
- **Implemented Method**: Unsupervised ensemble Isolation Forest (`sklearn.ensemble.IsolationForest``) calibrated on log-transformed sanctioned amounts ($\log(1 + \text{sanction_amount})$).
- **Hierarchical Peer Grouping Strategy**:
 1. **Primary Peer Group**: `(state, work_type)`` (State-level peer group).
 2. **Fallback Peer Group**: If primary peer group contains ≤ 15 historical works, fall back to national peer group `(work_type)``.
 3. **Insufficient Peer Data**: If national peer group contains ≤ 5 historical works, assign `severity = "INSUFFICIENT_PEER_DATA"`, `cost_anomaly_score = 0.0``, and record explanatory notice.
 - **Data Quality Exceptions**: If `sanction_amount`` is missing, zero, or negative, route work to `severity = "DATA_QUALITY_EXCEPTION"`, `is_data_quality_exception = True``, bypassing ML scoring.
 - **Score Calibration**: Raw Isolation Forest decision function scores are inverted and mapped to a continuous anomaly score $\in [0.0, 1.0]$ where higher values indicate higher deviation from the peer distribution.
 - **Severity Classification**:
 - `HIGH``: Score ≥ 0.85 (significant divergence from peer median).
 - `MEDIUM``: $0.70 \leq \text{Score} < 0.85$.
 - `LOW``: Score < 0.70 .
 - **Special Tags**: `"DATA_QUALITY_EXCEPTION"`, `"INSUFFICIENT_PEER_DATA"`.
 - **Explanation Generation**: Formulates natural language explanations: `"Sanctioned amount (X) is Y% above the peer group median (Z) across N peer projects at the [District/State/National] level."`

- **Limitations**: Does not predict future inflation; evaluates cross-sectional historical peer distribution only. Does not generate synthetic `expected\_cost` values.

5.2 Model 2: Potential Duplicate Work Detector (NLP Embeddings & Blocking)

- **Purpose**: Identify potential duplicate proposals, repeated asset allocations, or overlapping work descriptions across parliamentary sessions.
- **Candidate Blocking Mechanism**: To prevent $O(N^2)$ combinatorial explosion across 98,825 works, candidates are partitioned into spatial-temporal blocking windows: works are paired only within the same state and within adjacent recommendation/sanction time horizons. Canonical pair ordering enforces $\text{work_id_1} < \text{work_id_2}$ to eliminate redundant permutations and prevent self-pairing.
- **Multi-Signal Similarity Scoring**:

1. **Semantic Similarity** (S_{sem}): Cosine similarity of 384-dimensional dense text embeddings generated by `sentence-transformers/all-MiniLM-L6-v2` over normalized `work\_description`.
2. **Structural Similarity** (S_{struct}): Jaccard token overlap on normalized entity tokens.
3. **Amount Similarity** (S_{amt}): Financial ratio $\frac{\min(A_1, A_2)}{\max(A_1, A_2)}$.
4. **Date Proximity** (S_{date}): Exponential decay function over time difference $|\Delta \text{days}|$.

- **Composite Pair Score Formulation**:

$$\text{duplicate_score} = 0.50 \cdot S_{\text{sem}} + 0.20 \cdot S_{\text{struct}} + 0.15 \cdot S_{\text{amt}} + 0.15 \cdot S_{\text{date}}$$

- **Confidence Calibration**: Multiplier penalizing generic descriptions (e.g., "installation of solar street lights" with < 5 words).
- **Severity Classification**:
 - **HIGH**: $\text{duplicate_score} \geq 0.85$.
 - **REVIEW**: $0.70 \leq \text{duplicate_score} < 0.85$.
 - **LOW**: $\text{duplicate_score} < 0.70$.
- **Statutory Defensibility Notice**:

[!IMPORTANT]

The system explicitly classifies detected pairs as **POTENTIAL DUPLICATES REQUIRING HUMAN REVIEW**, and **NEVER** as confirmed fraud. Legitimate administrative reasons (e.g., multi-phase road work, identical street lights installed across neighboring wards) frequently produce high text similarity.

5.3 Model 3: Fund & Expenditure Anomaly Detector (Isolation Forest & Rules)

- **Purpose**: Detect irregular financial disbursement patterns, dormant project sanctions, and excessive vendor/contractor payment concentration.
- **Evaluated Features**:

1. **utilization_ratio**: $\frac{\text{total_disbursed_amount}}{\text{sanction_amount}}$.
2. **transaction_count**: Total disbursement voucher records in `work\_expenditures`.
3. **payment_concentration_hhi**: Herfindahl-Hirschman Index across payee vendors ($\sum_{i=1}^k s_i^2 \in [0.0, 1.0]$).
4. **days_to_first_disbursement**: Days elapsed between `sanction\_date` and first voucher `expenditure\_date`.

- **Implemented Architecture**: Isolation Forest trained on robust scaled disbursement features, combined with deterministic audit category assignment.
- **Audit Categories**:
 - `ACTIVE_EXPENDITURE`: Standard disbursements occurring according to project milestones.
 - `NORMAL_AWAITING_DISBURSEMENT`: Recently sanctioned works (<180\$ days) awaiting first contractor billing.
 - `DORMANT_SANCTION`: Works sanctioned >180\$ days prior with zero voucher entries recorded.
 - `STATUS_EXPENDITURE_MISMATCH`: Projects marked as `Completed` with 0% fund utilization, or ongoing projects whose cumulative disbursements exceed 100% of sanctioned limit.
- **Severity Classification**:
 - `HIGH`: Score ≥ 0.80 .
 - `MEDIUM`: $0.60 \leq \text{Score} < 0.80$.
 - `LOW`: Score < 0.60 .
- **Limitations**: Dependent on transaction vouchers being registered in portal data. Cash disbursements or offline payments are not visible to the engine.

5.4 Model 4: Statutory Delay Engine (Deterministic Rule-Based Implementation)

- **Engine Type**: **Strictly deterministic rule-based engine** codified from official statutory guidelines (NOT a machine learning model).
- **Governing Guidelines**: **MPLADS Guidelines 2023 (MoSPI)**.
- **Codified Statutory Rules**:
 1. **Recommendation-to-Sanction SLA (Para 3.12)**:
 - Statutory Window: **75 calendar days**.
 - Elapsed Days: $\text{rec_to_sanc_days} = \text{sanction_date} - \text{recommended_date}$.
 - Delay Calculation: $\text{rec_to_sanc_delay_days} = \max(0, \text{rec_to_sanc_days} - 75)$.
 - Severity: `HIGH` (>90\$ days overdue), `MEDIUM` (31 to 90\$ days), `LOW` (1 to 30\$ days), `NONE` (0\$ days).
 2. **Sanction-to-Completion SLA**:
 - Statutory Window: **365 calendar days (1 year)** standard completion threshold.
 - Elapsed Days: $\text{sanc_to_comp_days} = \text{completion_date} - \text{sanction_date}$.
 - Delay Calculation: $\text{sanc_to_comp_delay_days} = \max(0, \text{sanc_to_comp_days} - 365)$.
 - Severity: `HIGH` (>180\$ days overdue), `MEDIUM` (61 to 180\$ days), `LOW` (1 to 60\$ days), `NONE`.
 3. **Open Work Aging & Stalled Works**:
 - Evaluated on open works (`is_completed_flag = False` or `completion_date IS NULL`).
 - Elapsed Days: $\text{open_work_aging_days} = \text{reference_date} - \text{sanction_date}$.
 - Overdue Calculation: $\text{open_work_overdue_days} = \max(0, \text{open_work_aging_days} - 365)$.
 - Severity: `HIGH` (>365\$ days overdue), `MEDIUM` (181 to 365\$ days), `LOW` (1 to 180\$ days), `NONE`.
 - **Statutory Limitation (Rejection Communication SLA)**:

[!NOTE]

Para 3.12 of MPLADS Guidelines 2023 mandates that if a work is rejected, reasons must be communicated to the MP within **45 days**. Because the national portal dataset **does not capture rejection notification dates**, this SLA cannot be evaluated against historical data. The system explicitly records this statutory limitation rather than fabricating false violations.

5.5 Zero Composite Scoring Policy

The system maintains an absolute prohibition on composite fraud scores:

- **No Composite Score**: Each analytical module operates on distinct statistical baselines. Combining an Isolation Forest outlier score with an NLP cosine similarity and a statutory day count produces an uninterpretable, mathematically invalid composite.
- **Decoupled Dossier Presentation**: The investigation dossier (`/works/:id``) renders four independent profile cards side-by-side, enabling human auditors to evaluate cost, duplicate risk, financial pacing, and procedural delay separately.

6. Data Requirements

6.1 Source Data Acquisition & Lineage

The system ingests official MPLADS data spanning the 17th Lok Sabha, 18th Lok Sabha, and Rajya Sabha Sitting sessions:

- `Works Recommended`` (work ID, recommendation dates, MP name, constituency, category, description).
- `Works Sanctioned`` (sanction amount, sanction date, IDA implementing agency, state, district).
- `Works Completed`` (completion date, completion flag).
- `Expenditure on Completed and On-going Works`` (transaction dates, voucher IDs, vendor names, disbursed sums).

6.2 Canonical Normalization & Transformation Pipeline

Executed via ``data_pipeline/``:

1. **Work ID Normalization**: Strips tab characters (``\t``), whitespace, and normalizes delimiters into standard canonical strings (e.g., ``WS/MP18229/2026-2027/279619``).
2. **Amount Parsing**: Cleans Indian numbering conventions (lakhs, crores, commas) into standard ``NUMERIC(15, 2)`` floats.
3. **Date Standardization**: Parses multi-format strings (``DD/MM/YYYY``, ``YYYY-MM-DD``, ``DD-Mon-YYYY``) into ISO 8601 SQL ``DATE`` values.
4. **Header and Total Stripping**: Detects and purges summary lines (e.g., ``"Grand Total"`, `"Page Total"``).

6.3 Data Validation & Quality Rules

- Works with missing ``state`` or ``district`` are rejected during ingestion.
- Works with negative or zero ``sanction_amount`` are tagged with ``is_data_quality_exception = True`` and preserved in the works table to ensure total accountability, but excluded from ML training sets.

- Negative date deltas ($\text{\textit{\texttt{\text{completion_date}}} < \text{\textit{\texttt{\text{sanction_date}}}}$ or $\text{\textit{\texttt{\text{sanction_date}}} < \text{\textit{\texttt{\text{recommended_date}}}}$) are flagged as administrative logging errors.

...

7. Database Schema & Entity Specifications

7.1 Relational Schema Overview

The database layer consists of 7 normalized relational tables in PostgreSQL 17+:

$$+ \text{-----} +$$

users

$$+ \text{-----} +$$
$$+-----+$$

works +

$$+ \text{---} + \text{---} + \blacksquare$$

□ □

[illegible]

■ (1:1) ■ (1:1) ■ (1:1) ■ (1:N) ■ (1:N) ■ (1:N)

▼ ▼ ▼ ▼ ▼ ■

-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

cost_anomaly		fund_expendit.		delay		work_		duplicate_work_
_results		_results		_results		expenditures		_results

-----+

work_id_1, work_id_2

$$+ \text{-----} +$$

7.2 Table Definitions & Constraint Rules

Table 1: `works` (Master Works Registry)

Column Name	Data Type	Constraints	Description
`work_id`	`VARCHAR(64)`	PRIMARY KEY	Unique alphanumeric work identifier.
`house`	`VARCHAR(32)`	NULLABLE	Parliamentary house (Lok Sabha / Rajya Sabha).

`state`	`VARCHAR(100)`	NOT NULL, INDEX	State or Union Territory name.
`district`	`VARCHAR(100)`	NOT NULL, INDEX	Administrative district name.
`ida`	`VARCHAR(100)`	NULLABLE	Implementing District Authority.
`mp_name`	`VARCHAR(150)`	NULLABLE, INDEX	Hon'ble Member of Parliament recommending work.
`constituency`	`VARCHAR(150)`	NULLABLE	Parliamentary constituency name.
`constituency_or_term`	`VARCHAR(150)`	NULLABLE	Constituency identifier or Rajya Sabha term.
`work_category`	`VARCHAR(100)`	NULLABLE	Sectoral category (e.g., Roads, Drinking Water).
`work_type`	`VARCHAR(255)`	NULLABLE, INDEX	Specific sub-type of asset created.
`work_description`	`TEXT`	NULLABLE	Official narrative description of work.
`work_status`	`VARCHAR(100)`	NULLABLE, INDEX	Status: Recommended, Sanctioned, Ongoing, etc.
`sanction_amount`	`NUMERIC(15, 2)`	NULLABLE	Statutorily sanctioned capital in INR.
`sanction_date`	`DATE`	NULLABLE, INDEX	Date official administrative sanction issued.
`recommended_date`	`DATE`	NULLABLE	Date work recommended by Hon'ble MP.
`completion_date`	`DATE`	NULLABLE	Date work officially completed.
`amount_disbursed`	`NUMERIC(15, 2)`	NULLABLE	Total cumulative disbursement reported in portal.
`is_completed_flag`	`BOOLEAN`	DEFAULT FALSE	Boolean completion indicator.
`image_url`	`TEXT`	NULLABLE	URL to site photograph or document artifact.
`created_at`	`TIMESTAMP`	DEFAULT NOW()	System record creation timestamp.

Table 2: `cost\_anomaly\_results` (Model 1 Findings)

Column Name	Data Type	Constraints	Description
`work_id`	`VARCHAR(64)`	PRIMARY KEY, FK `works.work_id` (CASCADE)	Associated work identifier.
`cost_anomaly_score`	`DOUBLE PRECISION`	NOT NULL, INDEX	Calibrated anomaly score \$in [0.0, 1.0]\$.
`raw_anomaly_score`	`DOUBLE PRECISION`	NULLABLE	Raw Isolation Forest decision function output.
`severity`	`VARCHAR(32)`	NOT NULL, INDEX	HIGH, MEDIUM, LOW, or exception code.
`peer_group_used`	`VARCHAR(255)`	NULLABLE	Description of peer cluster used for training.
`peer_group_level`	`VARCHAR(64)`	NULLABLE	Resolution level: `STATE` or `NATIONAL`.

`peer_group_size`	`INTEGER`	NULLABLE	Sample count of historical projects in peer group.
`is_data_quality_exception`	`BOOLEAN`	DEFAULT FALSE	Flag indicating invalid/missing sanction amount.
`explanation`	`TEXT`	NULLABLE	Forensic rationale comparing cost to peer median.

Table 3: `duplicate\_work\_results` (Model 2 Findings)

Column Name	Data Type	Constraints	Description
`id`	`BIGSERIAL`	PRIMARY KEY	Unique candidate pair row identifier.
`work_id_1`	`VARCHAR(64)`	NOT NULL, INDEX, FK `works.work_id` (CASCADE)	Lexicographically smaller work ID.
`work_id_2`	`VARCHAR(64)`	NOT NULL, INDEX, FK `works.work_id` (CASCADE)	Lexicographically larger work ID.
`duplicate_score`	`DOUBLE PRECISION`	NOT NULL, INDEX	Weighted pair duplicate score \$in [0.0, 1.0]\$.
`severity`	`VARCHAR(32)`	NOT NULL, INDEX	Severity tier: `HIGH`, `REVIEW`, `LOW`.
`confidence`	`DOUBLE PRECISION`	NULLABLE	Confidence score discounting generic text.
`semantic_similarity`	`DOUBLE PRECISION`	NULLABLE	Dense cosine embedding similarity \$in [0, 1]\$.
`structural_score`	`DOUBLE PRECISION`	NULLABLE	Jaccard token overlap score \$in [0, 1]\$.
`amount_similarity`	`DOUBLE PRECISION`	NULLABLE	Sanctioned cost ratio \$in [0, 1]\$.
`date_proximity`	`DOUBLE PRECISION`	NULLABLE	Normalized date closeness score \$in [0, 1]\$.
`days_diff`	`INTEGER`	NULLABLE	Absolute days elapsed between project dates.
`is_same_mp`	`BOOLEAN`	NULLABLE	True if both works recommended by same MP.
`is_same_constituency`	`BOOLEAN`	NULLABLE	True if both works located in same constituency.
`explanation`	`TEXT`	NULLABLE	Natural language review justification.
Constraint	`UNIQUE`	`(work_id_1, work_id_2)`	Enforces unique undirected pair records.

Table 4: `fund\_expenditure\_results` (Model 3 Findings)

Column Name	Data Type	Constraints	Description
`work_id`	`VARCHAR(64)`	PRIMARY KEY, FK `works.work_id` (CASCADE)	Associated work identifier.
`fund_anomaly_score`	`DOUBLE PRECISION`	NOT NULL, INDEX	Calibrated disbursement score \$in [0.0, 1.0]\$.

`raw_score`	`DOUBLE PRECISION`	NULLABLE	Raw algorithm score.
`severity`	`VARCHAR(32)`	NOT NULL, INDEX	HIGH, MEDIUM, LOW.
`audit_category`	`VARCHAR(64)`	NULLABLE, INDEX	ACTIVE_EXPENDITURE, DORMANT_SANCTION, etc.
`total_disbursed_amount`	`NUMERIC(15, 2)`	NULLABLE	Reconciled voucher expenditure sum.
`utilization_ratio`	`DOUBLE PRECISION`	NULLABLE	Ratio of disbursed sum to sanctioned amount.
`transaction_count`	`INTEGER`	NULLABLE	Count of individual voucher payment lines.
`payment_concentration_hhi`	`DOUBLE PRECISION`	NULLABLE	Herfindahl-Hirschman Index \$in [0.0, 1.0]\$.
`days_to_first_disbursement`	`DOUBLE PRECISION`	NULLABLE	Days from sanction to first voucher date.
`anomaly_reasons`	`TEXT[]`	NULLABLE	Array of machine reason codes.
`explanation`	`TEXT`	NULLABLE	Detailed financial audit narrative.

Table 5: `delay\_results` (Model 4 Findings)

Column Name	Data Type	Constraints	Description
`work_id`	`VARCHAR(64)`	PRIMARY KEY, FK `works.work_id` (CASCADE)	Associated work identifier.
`delay_score`	`DOUBLE PRECISION`	NOT NULL, INDEX	Normalized delay score \$in [0.0, 1.0]\$.
`severity`	`VARCHAR(32)`	NOT NULL, INDEX	HIGH, MEDIUM, LOW, NONE.
`primary_delay_type`	`VARCHAR(64)`	NULLABLE, INDEX	RECOMMENDATION_SANCTION_DELAY, etc.
`active_delay_types`	`TEXT[]`	NULLABLE	Array of active statutory delay breaches.
`rec_to_sanc_days`	`INTEGER`	NULLABLE	Total days from recommendation to sanction.
`rec_to_sanc_delay_days`	`INTEGER`	NULLABLE	Days exceeding the 75-day statutory SLA.
`rec_to_sanc_severity`	`VARCHAR(32)`	NULLABLE	Specific recommendation SLA severity.
`sanc_to_comp_days`	`INTEGER`	NULLABLE	Total days from sanction to completion.
`sanc_to_comp_delay_days`	`INTEGER`	NULLABLE	Days exceeding the 365-day statutory threshold.
`sanc_to_comp_severity`	`VARCHAR(32)`	NULLABLE	Specific completion SLA severity.
`open_work_aging_days`	`INTEGER`	NULLABLE	Total elapsed days from sanction to reference date.
`open_work_overdue_days`	`INTEGER`	NULLABLE	Days open work has stalled past 365-day SLA.
`open_work_aging_severity`	`VARCHAR(32)`	NULLABLE	Specific stalled work severity.

`explanation`	`TEXT`	NULLABLE	Chronological SLA compliance breakdown.
---------------	--------	----------	---

Table 6: `work\_expenditures` (Voucher Transaction Lines)

Column Name	Data Type	Constraints	Description
`id`	`BIGSERIAL`	PRIMARY KEY	Unique voucher transaction ID.
`work_id`	`VARCHAR(64)`	NOT NULL, INDEX, FK `works.work_id` (CASCADE)	Foreign key to master work.
`expenditure_date`	`DATE`	NULLABLE, INDEX	Date voucher processed.
`vendor_name`	`VARCHAR(255)`	NULLABLE	Name of contractor / payee agency.
`fund_disbursed_amount`	`NUMERIC(15, 2)`	NULLABLE	Value of disbursement voucher in INR.
`payment_status`	`VARCHAR(64)`	NULLABLE	Transaction payment clearance status.

Table 7: `users` (Stakeholder Accounts & Jurisdictions)

Column Name	Data Type	Constraints	Description
`id`	`INTEGER`	PRIMARY KEY, AUTOINCREMENT	Unique user account ID.
`email`	`VARCHAR(255)`	UNIQUE, NOT NULL, INDEX	Official government email address.
`hashed_password`	`VARCHAR(255)`	NOT NULL	12-round bcrypt password hash.
`full_name`	`VARCHAR(150)`	NOT NULL	Full name and designation of officer.
`role`	`VARCHAR(32)`	NOT NULL, INDEX	MINISTRY, STATE_OFFICER, DISTRICT_OFFICER, MP.
`assigned_state`	`VARCHAR(100)`	NULLABLE	Bound State jurisdiction.
`assigned_district`	`VARCHAR(100)`	NULLABLE	Bound District jurisdiction.
`assigned_mp_name`	`VARCHAR(150)`	NULLABLE	Bound MP portfolio recommendation name.
`is_active`	`BOOLEAN`	DEFAULT TRUE, NOT NULL	Account activation status (revocation flag).
`created_at`	`TIMESTAMP TZ`	DEFAULT NOW()	Account registration timestamp.

8. API Interface Specifications

The backend exposes a standardized RESTful API under the prefix `/api/v1``.

8.1 Authentication Endpoints

- `POST /api/v1/auth/login``

- **Purpose**: Authenticate user and issue JWT token.
- **Auth**: None (Rate limited: 5 requests / minute / IP).
- **Body**: `{ "email": "user@mplads.gov.in", "password": "..."}`
- **Response**: `200 OK` $\rightarrow$ `TokenResponse` (`access_token`, `token_type: "bearer"`).
- **Errors**: `401 Unauthorized` (bad credentials/inactive), `422 Unprocessable` (schema error), `429 Too Many Requests`.
- **GET /api/v1/auth/me**
- **Purpose**: Retrieve authenticated user profile and jurisdiction bindings.
- **Auth**: Bearer token required.
- **Response**: `200 OK` $\rightarrow$ `UserRead` (`id`, `email`, `full_name`, `role`, `assigned_state`, `assigned_district`, `assigned_mp_name`, `is_active`).
- **Errors**: `401 Unauthorized` (expired or invalid token).
- **GET /api/v1/auth/users**
- **Purpose**: List all registered users and administrative assignments.
- **Auth**: Bearer token required. Role whitelist: `MINISTRY`.
- **Response**: `200 OK` $\rightarrow$ `List[UserRead]`.
- **Errors**: `403 Forbidden` (non-ministry callers).
- **POST /api/v1/auth/users**
- **Purpose**: Provision a new officer account with jurisdictional constraints.
- **Auth**: Bearer token required. Role whitelist: `MINISTRY`.
- **Body**: `UserCreate` (`email`, `password`, `full_name`, `role`, `assigned_state`, `assigned_district`, `assigned_mp_name`).
- **Response**: `201 Created` $\rightarrow$ `UserRead`.
- **Errors**: `400 Bad Request` (email already exists), `403 Forbidden` (non-ministry callers).

8.2 Works Master Registry Endpoints

- **GET /api/v1/works**
- **Purpose**: Query works catalog with multi-attribute filtering and pagination.
- **Auth**: Bearer token required (scoped by caller's jurisdiction).
- **Query Parameters**: `page` (default 1), `page_size` (default 20, max 100), `state`, `district`, `mp_name`, `house`, `work_category`, `work_status`, `min_sanction_amount`, `max_sanction_amount`, `search`.
- **Response**: `200 OK` $\rightarrow$ `PaginatedResponse[WorkListItem]`.
- **GET /api/v1/works/{work_id}**
- **Purpose**: Retrieve complete investigation dossier for single work.
- **Auth**: Bearer token required. Enforces `verify_work_jurisdiction`.
- **Response**: `200 OK` $\rightarrow$ `WorkDetail` (metadata, expenditures list, independent risk profiles).

- **Errors**: `404 Not Found` (work does not exist), `403 Forbidden` (work exists but belongs outside caller's jurisdiction).
- **GET /api/v1/works/meta/filters**
- **Purpose**: Provide distinct dynamic filter options for UI dropdowns.
- **Auth**: Bearer token required.
- **Response**: `200 OK` \$\rightarrow\$ `FilterOptionsResponse` (states, districts, houses, categories, statuses, severities).

8.3 Analytical Module Endpoints

- **GET /api/v1/analytics/cost-anomalies**
- **Query Parameters**: `severity`, `min\_score`, `state`, `district`, `page`, `page\_size`.
- **Response**: `200 OK` \$\rightarrow\$ `PaginatedResponse[CostAnomalyItem]`.
- **GET /api/v1/analytics/cost-anomalies/{work_id}**
- **Response**: `200 OK` \$\rightarrow\$ `CostAnomalyDetail`.
- **GET /api/v1/analytics/duplicate-works**
- **Query Parameters**: `severity`, `min\_duplicate\_score`, `is\_same\_mp`, `is\_same\_constituency`, `page`, `page\_size`.
- **Response**: `200 OK` \$\rightarrow\$ `PaginatedResponse[DuplicatePairItem]`.
- **GET /api/v1/analytics/duplicate-works/pairs/{work_id}**
- **Response**: `200 OK` \$\rightarrow\$ `WorkDuplicateLookupResponse` (`work\_id`, `total\_flagged\_pairs`, `pairs`).
- **GET /api/v1/analytics/fund-anomalies**
- **Query Parameters**: `severity`, `audit\_category`, `min\_score`, `min\_utilization`, `max\_utilization`, `state`, `district`, `page`, `page\_size`.
- **Response**: `200 OK` \$\rightarrow\$ `PaginatedResponse[FundAnomalyItem]`.
- **GET /api/v1/analytics/fund-anomalies/{work_id}**
- **Response**: `200 OK` \$\rightarrow\$ `FundAnomalyDetail`.
- **GET /api/v1/analytics/delays**
- **Query Parameters**: `severity`, `primary\_delay\_type`, `min\_days\_overdue`, `state`, `district`, `page`, `page\_size`.
- **Response**: `200 OK` \$\rightarrow\$ `PaginatedResponse[DelayItem]`.
- **GET /api/v1/analytics/delays/{work_id}**
- **Response**: `200 OK` \$\rightarrow\$ `DelayDetail`.

8.4 Governance Summary Endpoints

- **GET /api/v1/analytics/district-summary**
- **Query Parameters**: `state` (optional), `limit` (default 50).
- **Response**: `200 OK` \$\rightarrow\$ `List[DistrictSummaryItem]`.
- **GET /api/v1/analytics/mp-summary**
- **Query Parameters**: `state` (optional), `house` (optional), `limit` (default 50).

- **Response**: `200 OK` \$\rightarrow\$ `List[MPSummaryItem]`.

8.5 Health & Diagnostics Endpoint

- **GET** /api/v1/health
- **Purpose**: System health check, PostgreSQL connectivity ping, latency benchmark.
- **Auth**: None (Public).
- **Response**: `200 OK` \$\rightarrow\$ `HealthCheckResponse` (`status`, `database`, `db\_latency\_ms`, `total\_works`, `version`).

9. Security & Access Control Requirements

+-----+

SECURITY & JURISDICTIONAL ENFORCEMENT

+-----+

Threat / Vulnerability	Implemented Defense Mechanism
------------------------	-------------------------------

+-----+

Credential Compromise / Brute Force	Bcrypt (12 rounds) + SlowAPI 5/min
User Enumeration / Timing Attacks	Dummy equalizing bcrypt computation
Session Tampering	Signed HMAC-SHA256 JWTs (60m expiry)
Horizontal Data Leakage	Server-side jurisdictional SQL predicate injection on every query
Cross-Jurisdiction Snooping	403 Redaction on foreign work cards
Immediate Account Revocation	Live DB check in auth dependency

+-----+

1. **Cryptographic Password Storage**:

Passwords are never stored in plaintext. Hashed exclusively using `bcrypt` with cost factor 12.

2. **Timing Attack & User Enumeration Mitigation**:

When a non-existent email is submitted to `/auth/login`, the system computes a pre-calculated 12-round dummy bcrypt verification (`verify\_dummy\_password()`), ensuring uniform response times (~90 ms) and defeating timing analysis.

3. **Short-Lived Signed Bearer Tokens**:

JWT access tokens are cryptographically signed with HMAC-SHA256 and expire after 60 minutes.

4. **Server-Side Jurisdictional Predicate Injection**:

Client requests cannot bypass geographic boundaries. SQL queries automatically inject mandatory jurisdictional filters matching the verified token claims.

5. **Collection Scoping vs. Single-Resource 403 Enforcement**:

- Collection endpoints (`/works`, `/analytics/\*`) silently filter out out-of-scope records, returning `200 OK` with only authorized items.
- Direct resource lookups (`/works/{id}`) explicitly return `403 Forbidden` if the requested resource exists but belongs to an unauthorized jurisdiction.

6. **Cross-Jurisdictional Duplicate Protection**:

When a duplicate pair spans two different jurisdictions, an officer can view the pair score and their own work, but details of the foreign work are masked with an official 403 placeholder card.

10. User Interface Requirements

The user interface is engineered as a responsive, modern single-page application:

1. **Authentication Page** (`Login.tsx`):

- Dual authentication modality: standard email/password input form + 1-click canonical demonstration switcher.
- Displays official MoSPI / SIH 2026 branding and security disclosures.

2. **Global Shell Layout** (`AppLayout.tsx`):

- **Header**: Live API health pulse, active jurisdictional scope badge (`MINISTRY`, `STATE\_OFFICER`, `DISTRICT\_OFFICER`, `MP`), demo role switcher, and user logout action.
- **Sidebar**: Two-tier navigation grouping:
 - **Core Operations**: Dashboard, Works Master Registry, District Summaries, MP Summaries.
 - **Independent Audits (4)**: Cost Anomalies, Duplicate Works, Fund Anomalies, Statutory Delays.
 - **System**: User Management (Ministry only).
- **RateLimitBanner**: Floating countdown banner activated on HTTP 429 errors.

3. **Role-Adaptive Dashboard** (`Dashboard.tsx`):

- Dynamically selects and renders one of 4 layout templates based on `user.role`.

4. **Data Presentation Tables** (`DataTable.tsx`):

- Standardized layout with sticky headers, zebra-striping, pagination controls, and status badges.

5. **Severity Badges** (`Badge.tsx`):

- Standardized visual color-coding: `HIGH` (Rose/Red), `MEDIUM` / `REVIEW` (Amber/Yellow), `LOW` (Emerald/Green), `DATA\_QUALITY\_EXCEPTION` (Purple).

6. **Work Investigation Dossier** (`WorkDetail.tsx`):

- Multi-tab layout featuring: Overview & Dates, Financial Allocations, Voucher Receipts Table, and 4 Decoupled Audit Profile Cards.

7. **Side-by-Side Duplicate Comparison Tool** (`DuplicateComparison.tsx`):

- Dual-card visual diff view highlighting overlapping phrases, date proximity, and cross-district redaction placeholders.

11. Non-Functional Requirements

11.1 Performance & Latency

- **API Response Latency**: 95% of paginated queries on the 98,825-record works catalog shall return within **<500 ms** under standard database connectivity.
- **Health Check Ping**: Real-time database latency telemetry shall execute in **<1,500 ms**.
- **Frontend Build Size**: Compressed production bundle shall remain **<1.0 MB** gzipped (`dist/assets/index.js` currently 225 kB gzip).
- **Client Render Speed**: Initial route render and dashboard hydration shall complete in **<300 ms**.

11.2 Scalability & Concurrency

- **Stateless Application Layer**: The FastAPI backend maintains zero server-side session state, enabling horizontal auto-scaling behind reverse proxies.
- **Database Connection Pooling**: PostgreSQL connections managed through Supabase PgBouncer pooler.

11.3 Availability & Reliability

- **System Uptime Target**: 99.9% availability during evaluation windows.
- **Degraded Network Handling**: Frontend client gracefully captures network dropouts, rendering non-blocking error banners and preserving cached TanStack data.

11.4 Explainability & Auditability

- **Forensic Justifications**: Every anomaly flagged by Models 1–4 must provide a human-readable explanatory sentence detailing why it was flagged.
- **Audit Trail**: All voucher lines maintain source transaction dates and vendor names.

12. Error Handling & Edge Cases

Scenario / Edge Case	Trigger Condition	System Behavior & Response Code
Invalid Password	Incorrect password on login	Returns <code>401 Unauthorized</code> : <code>"Incorrect email or password"</code> .
Non-Existent Email	Email not in database	Executes dummy bcrypt verification (~90ms) and returns <code>401 Unauthorized</code> .
Deactivated Account	<code>is_active = False</code> in DB	Rejects with <code>401 Unauthorized</code> : <code>"User account is inactive"</code> .
Expired JWT Token	Token older than 60 mins	Interceptor clears <code>localStorage</code> and redirects browser to <code>/login</code> .

Rate Limit Triggered	$\$>5\$$ login attempts in 1 min	Returns `429 Too Many Requests`. UI displays floating countdown banner.
Unauthorized Single Work	Accessing `/works/{id}` out of scope	Aborts with `403 Forbidden`: <code>"Access to work outside assigned jurisdiction is forbidden"</code> .
Conflicting Filters	User queries out-of-scope state	Returns `200 OK` with empty list `[]` (silent collection scoping).
Cross-Jurisdiction Duplicate	Pair spans two different districts	Pair visible in list; unauthorized work details redacted with 403 card.
Negative Sanction Amount	<code>`sanction_amount <= 0`</code>	Flagged as `DATA_QUALITY_EXCEPTION`. Cost score set to 0.0.
Missing Peer Group Data	$\$<5\$$ historical peer works in India	Flagged as `INSUFFICIENT_PEER_DATA`. Cost score set to 0.0.
Zero-Disbursement Work	Sanctioned $\$>180\$$ days with 0 vouchers	Flagged as `DORMANT_SANCTION` under Model 3 with High severity.

13. Requirements Traceability Matrix

Req ID	Requirement Description	Architectural Component	Source Code Implementation	Verification Test File
FR-01	User Authentication & JWT Issuance	Auth Router / Security	`api/routers/auth.py`, `api/auth/security.py`	`tests/test_auth_rbac.py`
FR-02	Real-Time Profile & User Verification	Auth Dependency	`api/auth/dependencies.py`	`tests/test_auth_rbac.py`
FR-03	Server-Side Jurisdictional Scoping	SQL Predicate Injector	`api/auth/scoping.py`	`tests/test_auth_rbac.py`
FR-04	Ministry National Dashboard	Presentation Layer	`frontend/src/pages/Dashboard.tsx`	End-to-end proxy tests
FR-05	State Officer Dashboard	Presentation Layer	`frontend/src/pages/Dashboard.tsx`	End-to-end proxy tests
FR-06	District Officer Triage Dashboard	Presentation Layer	`frontend/src/pages/Dashboard.tsx`	End-to-end proxy tests
FR-07	MP Portfolio Lifecycle Dashboard	Presentation Layer	`frontend/src/pages/Dashboard.tsx`	End-to-end proxy tests
FR-08	Works Registry & Multi-Search	Works Router / UI	`api/routers/works.py`, `WorksRegistry.tsx`	`tests/test_api.py`
FR-09	Investigation Dossier	Works Router / UI	`api/routers/works.py`, `WorkDetail.tsx`	`tests/test_api.py`
FR-10	Model 1: Cost Anomaly Console	Analytics / UI	`api/routers/cost_anomalies.py`, `CostAnomalies.tsx`	`tests/test_model1_cost_anomaly.py`
FR-11	Model 2: Duplicate Work Console	Analytics / UI	`api/routers/duplicate_works.py`, `DuplicateWorks.tsx`	`tests/test_model2_duplicate_work.py`
FR-12	Model 2: Pair Comparison Flow	Analytics / UI	`api/routers/duplicate_works.py`, `DuplicateComparison.tsx`	`tests/test_model2_duplicate_work.py`

FR-13	Model 3: Fund Anomaly Console	Analytics / UI	`api/routers/fund_anomalies.py`, `FundAnomalies.tsx`	`tests/test_model3_fund_expenditure.py`
FR-14	Model 4: Statutory Delay Console	Rule Engine / UI	`api/routers/delays.py`, `StatutoryDelays.tsx`	`tests/test_delay_rules.py`
FR-15	District Summary Aggregations	Summary Router / UI	`api/routers/summaries.py`, `DistrictSummary.tsx`	`tests/test_api.py`
FR-16	MP Summary Aggregations	Summary Router / UI	`api/routers/summaries.py`, `MPSummary.tsx`	`tests/test_api.py`
FR-17	Ministry User Administration	Auth Router / Admin UI	`api/routers/auth.py`, `AdminUsers.tsx`	`tests/test_auth_rbac.py`
FR-18	Health Telemetry & DB Ping	Health Router / UI	`api/routers/health.py`, `Header.tsx`	`tests/test_api.py`

14. Known System Limitations

1. ****Absence of Rejection Date Milestone****:

The current national portal dataset records only approved and sanctioned works; proposal rejection dates and formal rejection notifications are unrecorded. Consequently, the 45-day rejection communication requirement (MPLADS Guidelines 2023 Para 3.12) cannot currently be evaluated.

2. ****Offline Batch Model Training****:

The Isolation Forest and NLP vector caches are generated via offline batch pipelines. New works ingested into the database are evaluated when the pipeline is executed; real-time on-the-fly model re-training is not implemented.

3. ****Absence of Direct Contractor Tax Identifiers (GSTIN/PAN)****:

Vendor tracking in Model 3 relies on string-normalized `vendor\_name` attributes in the expenditure logs. The absence of corporate tax identifiers in the source dataset limits vendor network entity resolution across spelling variations.

15. Future Enhancements (Deferred Scope)

1. ****OCR & Computer Vision Verification****:

Integration of OCR and computer vision pipelines to inspect uploaded project completion certificates and geo-tagged photographs against work descriptions.

2. ****Automated Contractor Entity Resolution****:

Fuzzy clustering and PAN/GSTIN integration to detect shell companies and multi-district contractor cartels.

3. ****Public RTI & Transparency Portal****:

Citizen-facing, read-only transparency portal providing open access to constituency asset maps without authentication.

16. Acceptance Criteria & Test Verification

All 18 functional requirements and security invariants have been empirically verified:

1. **Full Repository Regression**:

- `pytest tests/ -v``: **85 of 85 tests PASSED (100% pass rate)**.
- API contract tests: 15 / 15 passed.
- RBAC & security tests: 19 / 19 passed.
- Machine learning & rule tests: 41 / 41 passed.
- Data pipeline tests: 10 / 10 passed.

2. **Frontend Production Build**:

- ``npm run build`` in ``frontend/``: Compiled in **2.46 seconds** with **zero TypeScript or bundle errors**.

3. **Zero Composite Score Validation**:

- Verified that ``works`` and ``work_details`` APIs contain independent objects for each model without synthetic combined risk formulas.

End of Software Requirements Specification — AI-Powered MPLADS Monitoring & Anomaly Detection System (SIH 2026 PS 26102)